

The Sixbell-Dev Way (TSDW)

Cheat-Sheet Operativo — Greenfield y Brownfield

Referencia rápida para trabajar con OpenSpec, Kiro Steering, Skills, Hooks y templates oficiales.

Reglas de oro

- OpenSpec es la fuente de verdad del cambio.
- No implementar cambios relevantes sin aprobación humana explícita.
- No usar el chat como fuente de verdad del cambio.
- Kiro es el entorno principal de ejecución.
- TSDW usa el schema sixbell-governed.
- Seguridad, trazabilidad, documentación y costo importan siempre.
- AWS-first salvo excepción aprobada.

Comandos base

Los comandos mínimos que deberías recordar en cualquier repositorio TSDW

Bootstrap del repositorio

```
npm run bootstrap:openspec  
npm run bootstrap:verify
```

Scripts del template fullstack

```
npm run validate:format  
npm run validate:lint  
npm run test:unit  
npm run test:smoke
```

OpenSpec (perfil core)

```
/opsx:propose  
/opsx:explore  
/opsx:apply  
/opsx:archive
```

Si más adelante habilitas workflow expandido

```
/opsx:new  
/opsx:continue  
/opsx:ff  
/opsx:verify  
/opsx:sync  
/opsx:archive
```

Greenfield — Flujo recomendado

Del template al archive del cambio

A) Crear el repositorio

- Crear repo desde template oficial (fullstack, frontend o backend).
- Abrir el repo en Kiro.
- Ejecutar bootstrap de OpenSpec y verificación.

```
npm run bootstrap:openspec
npm run bootstrap:verify
```

Prompt sugerido

Resume este repositorio según TSDW y explícame:

1. la estructura del proyecto,
2. las capabilities actuales en openspec/specs,
3. los hooks locales disponibles,
4. el flujo correcto para hacer el primer cambio.

B) Definir el primer cambio

```
/opsx:propose <descripción-del-cambio>
```

Prompt sugerido

Usa sixbell-openspec-governance para validar este cambio y dime:

1. qué artifacts faltan,
2. qué decisiones están subespecificadas,
3. cuál debería ser el siguiente paso válido.

C) Refinar artifacts

- Revisar proposal.md, delta specs, design.md, review.md y tasks.md.
- Asegurar que objetivos, alcance, riesgos, diseño y review están explícitos.

Prompt sugerido

Refina el design.md del cambio asegurando que incluya:

- contexto,
- goals / non-goals,
- decisiones clave,
- trade-offs,
- rollout / rollback,
- preguntas abiertas.

D) Revisión antes de implementar

- Usar la skill sixbell-openspec-governance.
- Si el cambio toca arquitectura, activar architecture-review-manual.
- Si toca seguridad/datos/auth, activar security-pre-commit-review.

Prompt sugerido

Usa sixbell-openspec-governance sobre el cambio activo y dime si está listo para implementación.

Quiero:

- readiness status,
- artifacts faltantes,
- blockers,
- warnings,
- siguiente paso recomendado.

E) Implementación

- Comando principal: `/opsx:apply`
- Hooks automáticos: `spec-gate-before-apply`, `format-lint-on-save`, `unit-test-on-save`, `api-doc-sync`.

```
/opsx:apply
```

Prompt sugerido

Implementa solo lo que está cubierto por los tasks actuales.
Si detectas que el diseño o las specs quedaron desalineadas con la implementación, detente y propón la actualización del artifact correspondiente antes de seguir.

F) Validación local

- Correr formato, lint, unit y smoke.
- Usar `sixbell-pr-review` para evaluar readiness.

```
npm run validate:format
npm run validate:lint
npm run test:unit
npm run test:smoke
```

Prompt sugerido

Usa `sixbell-pr-review` sobre el cambio actual y entrégame:

- PR readiness status,
- blocking issues,
- warnings,
- suggested follow-up items.

G) Preparar PR

- Revisar PR template, `review.md`, tasks completas, docs alineadas y smoke tests.
- Completar el PR basándose en el OpenSpec change activo.

Prompt sugerido

Ayúdame a completar el PR template de este repositorio basándote en:

- el OpenSpec change activo,
- el `review.md`,
- el riesgo identificado,
- la validación ejecutada,
- y la documentación actualizada.

H) Cierre del cambio

- Archivar solo cuando tareas, validación y documentación estén alineadas.

```
/opsx:archive
```

Prompt sugerido

Antes de archivar, confirma que:

1. las tareas están completas,
2. el cambio fue validado,
3. la documentación quedó alineada,
4. las specs activas pueden convertirse en nueva fuente de verdad.

Brownfield — Flujo recomendado

Primero capturar verdad actual, luego cambiar con rigor

A) Adoptar TSDW en el repo existente

- Añadir/normalizar: .kiro/, .github/, openspec/, docs/, adr/, api/, tests/.
- Ejecutar bootstrap:openspec y bootstrap:verify.

```
npm run bootstrap:openspec
npm run bootstrap:verify
```

Prompt sugerido

```
Analiza este repositorio existente y dime:
1. qué partes del estándar TSDW ya están presentes,
2. qué falta para dejarlo OpenSpec-native,
3. qué capabilities existentes debería documentar primero en openspec/specs.
```

B) Capturar la verdad actual

- Poblar openspec/specs/ con las capabilities más importantes del sistema actual.
- No intentar documentar todo de una vez; empezar por lo crítico y lo que se va a tocar.

Prompt sugerido

```
Crea o refina el spec de esta capability legacy para representar el comportamiento actual
aprobado del sistema, no un comportamiento deseado futuro.
```

C) Crear y refinar un cambio brownfield

```
/opsx:propose <cambio-en-sistema-existente>
```

Prompt sugerido

```
Este es un cambio brownfield. Refuerza el cambio con foco en:
- comportamiento actual afectado,
- delta specs,
- compatibilidad,
- rollback,
- migración,
- y riesgo transversal.
```

D) Aumentar rigor si hace falta

- Usar con más frecuencia: sixbell-openspec-governance, sixbell-security-review, architecture-review-manual, security-pre-commit-review.
- Subir a medium/high si hay migración, arquitectura, blast radius, compatibilidad o datos sensibles.

```
Este cambio ocurre en un sistema legacy. Evalúa si debe subir de rigor (medium/high)
considerando:
- arquitectura,
- migración,
- blast radius,
- compatibilidad,
- datos sensibles,
- rollback.
```

Skills y Hooks

Cuándo usar cada elemento del framework

Skills Sixbell propias

- **sixbell-openspec-governance**: validar readiness antes de implementar.
- **sixbell-pr-review**: revisar readiness antes de PR.
- **sixbell-security-review**: revisar seguridad en cambios sensibles o de mayor riesgo.

Usa sixbell-openspec-governance sobre el cambio activo y dime si está listo para implementación.

Usa sixbell-pr-review sobre el cambio actual y entrégame un PR readiness status.

Usa sixbell-security-review sobre este cambio y dame findings con severidad, blocking/warning y remediación.

Skills técnicas externas (opcionales)

- vercel-composition-patterns: React composition y APIs de componentes.
- vercel-react-best-practices: React/Next performance.
- vercel-react-native-skills: React Native / Expo.
- web-design-guidelines: auditoría UI/UX/accesibilidad.

Hooks

- **Automáticos**: spec-gate-before-apply, format-lint-on-save, unit-test-on-save, api-doc-sync.
- **Manuales**: architecture-review-manual, security-pre-commit-review, smoke-on-demand.

Secuencia ideal resumida — Greenfield

- Template → bootstrap:openspec → bootstrap:verify
- /opsx:propose → governance validation → design/review/tasks
- human approval → /opsx:apply → tests + smoke + PR review → PR → /opsx:archive

Secuencia ideal resumida — Brownfield

- Adopt TSDW → bootstrap:openspec → bootstrap:verify
- capturar current capabilities → /opsx:propose
- delta specs + design + review → risk/security/architecture review → human approval
- /opsx:apply → tests + smoke + PR review → PR → /opsx:archive

Frases rápidas para copiar y pegar

Prompts útiles para el trabajo diario

Resume este repositorio según TSDW y explícame el flujo correcto para trabajar en él.

/opsx:propose <describe el cambio aquí>

Usa sixbell-openspec-governance sobre el cambio activo y dime si está listo para implementación.

Usa sixbell-security-review sobre este cambio y dame blockers, warnings y remediaciones.

Usa sixbell-pr-review sobre el cambio actual y entrégame un PR readiness status.

Refina este design.md según TSDW, asegurando contexto, decisiones, trade-offs, rollout/rollback y preguntas abiertas.

Este es un cambio brownfield. Refuerza el rigor del análisis considerando compatibilidad, migración, rollback y blast radius.

Ayúdame a completar el PR template usando el OpenSpec change activo, el review.md y los resultados de validación.

Piensa TSDW así

- OpenSpec: define el cambio.
- Steering: define las reglas del mundo.
- Skills: te ayudan a operar el proceso.
- Hooks: te recuerdan o fuerzan disciplina en momentos clave.
- Governance docs: te dan criterio humano y trazabilidad institucional.

Si usas TSDW bien, el cambio no vive en el chat: vive en artifacts, decisiones, revisión y evidencia.